MalAI

Group Number: 20

Goals

Project Purposes

MalAI will reduce the time it takes for a user to reverse engineer an executable by providing an improved decompilation process

Goal Statements:

Malware analysis tool that utilizes reverse engineering and artificial intelligence models to analyze executable files and detect malware

What is MalAI?

- MalAI is an AI-powered Ghidra plugin that enhances binary decompilation
- It integrates work from LLM4Decompile into a real-world workflow
- Trained on ASM-Source pair datasets such as Exebench and DecompileBench
- Uses an LLM to transform assembly into more readable and meaningful C-Code
- Aims to reduce time taken to reverse engineer an executable
- Includes AI-based malware detection and injection as extended functionalities

Involved People

Project Advisor:

Dr. Abuaitah

Group Members

Long Nguyen

- Email: nguye2l6@mail.uc.edu

Luqman Al Hasni

- Email: alhasnld@mail.uc.edu

Project Impact

Research:

- Providing a decompilation model that is inspired by an ongoing research paper (LLM4Decompile)

Ghidra:

- Enhancing Ghidra's Code Browser results-
- Enabling users to detect malware through Ghidra via a simplified process

LLM4Decompile

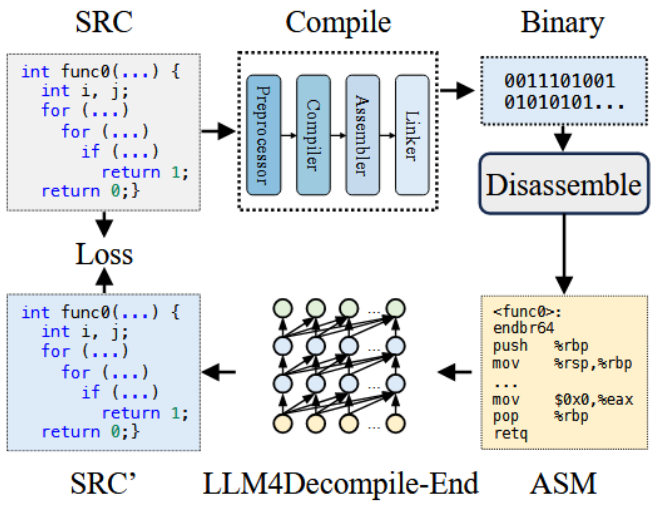


Figure 2: *End2end-Decompile* framework. The source code (SRC) is compiled to binary, disassembled to assembly instructions (ASM), and decompiled by LLM4Decompile to generate SRC'. Loss is computed between SRC and SRC' for training.

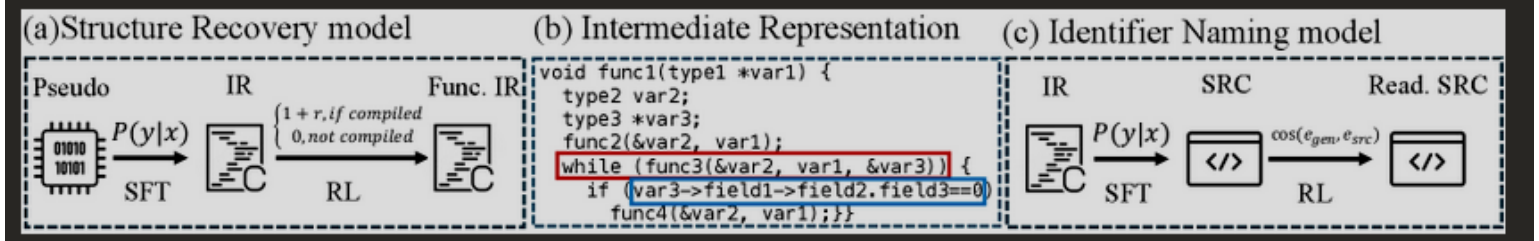


Figure 2: Overview of *SK²Decompile* with two-phase decompilation process (a) Structure Recovery and (c) Identifier Naming. Obfuscated source code (b) serves as the Intermediate Representation (IR) connecting these two phases. For each model, a supervised fine-tuning (SFT) process is performed followed by Reinforcement Learning (RL) with its respective reward.

LLM4Decompile – Cont'

Source Code

```
int func0(float num[], int size,
  float threshold) {
  int i, j;
  for (i = 0; i < size; i++)
    for (j = i + 1; j < size; j++)
      if (fabs(num[i] - num[j])
          < threshold)
          return 1;
  return 0;}
```

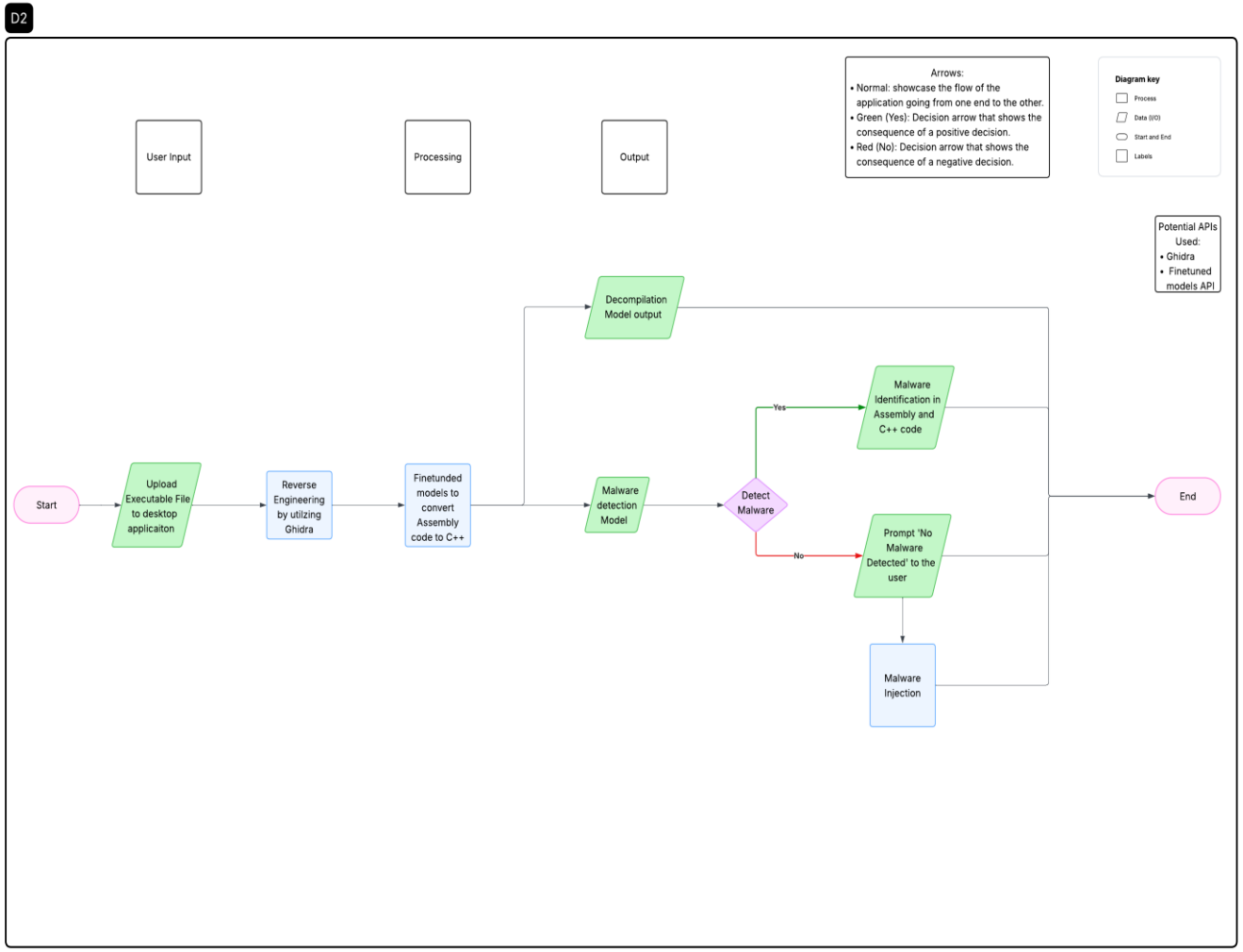
LLM4Decompile-End-6.7B

```
int func0(float *x, int n,
  float eps){
  int i, j;
  for (i = 0; i < n; i++)
    for (j = i + 1; j < n; j++)
      if (fabsf(x[i] - x[j]) < eps)
        return 1;
  return 0;}
```

Ghidra

```
undefined4 func0(float param_1,long param_2,int param_3){
  int local_28;
  int local_24;
  local_24 = 0;
  do {
    local_28 = local_24;
    if (param_3 <= local_24) {
      return 0;}
  while (local_28 = local_28 + 1, local_28 < param_3) {
    if ((double)((ulong)(double)(*(float *)(param_2 + (long)local_24 * 4) -
        *(float *)(param_2 + (long)local_28 * 4)) &
        SUB168(_DAT_00402010,0)) < (double)param_1) {
      return 1;}}
    local_24 = local_24 + 1;
  } while( true );}
```

MalAI Flowchart



User stories

- As a malware analyst, I reverse engineer countless executables through my career. However, the process of reverse engineering can be time-consuming as the current decompilation processes provides hard-to-read c-code. Therefore, I utilize MalAI to speed up my analysis by generating more readable c-code, allowing me to determine whether an executable is malware more quickly.

User Stories: Cont’

- As a software developer, I often download many executable files for various reasons (testing applications, downloading coding-related software, etc.). However, this habit opens a lot of doors for my computer to get harmed. Therefore, I utilize MalAI to identify any malware in the executable files I intend to use.

Technologies

Ghidra – MalAI is integrated within the Ghidra ecosystem as a plugin, serving as an improvement over Ghidra's built-in decompilation processes

LLMs – MalAI leverages LLMs as a tool to refine assembly-level code into more readible C-code

QLoRA Finetuning – MalAI builds off Deepseek-Coder 7B by training on additional ASM-Source pairs to improve decompilation capabilities

LLM4Decompile – Two implementation of enhanced reverse engineering – From scratch model and one that refines Ghidra’s decompilation output

SK2Decompile – Two phase decompilation process that seperates code into skeleton(skeleton) and skin(semantics)

CodableLLM – Python framework for creating code datasets for LLM training

Milestones

- Milestones:
  - Research available solutions
  - Chose potential datasets
  - Explore datasets performance
  - Train LLM via chosen datasets
  - Build the foundation of Ghidra plugin
  - Integrating finetuned LLM with Ghidra plugin
  - Demo MalAI
  - Provide malware analysis solution via Ghidra plugin
  - Provide malware Injection solution via Ghidra plugin
  - Develop a landing webpage for the MalAI

Challenges

Research-Driven Development

LLM-based decompilation is a ongoing research area. It is difficult to stay up to date with cutting-edge model architectures and training

Computation

Finetuning LLMs for decompilation on a similar scale similar to SK2Decompile and LLM4Decompile requires considerable computational resources

Model Accuracy/Readability

Deepseek Coder is a good basis for decompilation but doesn't guarantee accuracy. Ghidra decompilation is always accurate but not always readable. It was challenging to ensure both metrics were being met while training

Ghidra Integration

Familiarity with Ghidra's environment and plugin development was a constraint.

Artificial intelligence

Limited background in LLM training and the process of finetuning.

Major Project Constraints

ETHICAL

SECURITY

LEGAL

Demo at EXPO

Use a monitor to showcase:

- The MalAI Ghidra Plugin output with a test executable file/files to demonstrate the improved decompilation results.
- The Integration of other AI models (malware detection and injection) within Ghidra by providing the user with simpler ways to analyze executable file.

End

References

- Tan, H., Luo, Q., Li, J., & Zhang, Y. (2024). LLM4Decompile: Decompiling binary code with large language models. Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 3473–3487. https://doi.org/10.18653/v1/2024.emnlp-main.203
- Tan, H., Li, W., Tian, X., Wang, S., Liu, J., Li, J., & Zhang, Y. (2025, September 26). *SK2Decompile: LLM-based two-phase binary decompilation from skeleton to skin*. arXiv.org. https://arxiv.org/abs/2509.22114